

MENTOR-RL Methods

Peter Kruse

June 2026

0.1 Introduction

Mechanistic interpretation is the discovery of relationships between molecular entities such as genes, proteins, and RNA. Systems biology approaches this by studying the interactions between those entities rather than their individual properties [13, 3], seeking to identify *modules*, groups of functionally related entities, and to characterize the mechanisms underlying their behavior. Because these interactions span multiple scales and regulatory layers, modern analysis relies on multiplex representations that hold the same genes across several types of edge [2, 22, 10], over which propagation methods such as random walk with restart (RWR) identify functionally related neighborhoods [16, 9]. Module-level analysis has become foundational for identifying disease mechanisms, prioritizing drug targets, and interpreting polygenic phenotypes [21], and methods such as iterative random forest (iRF) [4, 8] have accelerated it by constructing biologically informed networks that scientists explore to generate hypotheses.

As these high-throughput and algorithmic methods increase the resolution and scope of networks, biological interpretation has become increasingly mediated by large, multi-scale graph structures. To comprehensively examine these large networks and supporting databases, mechanistic exploration requires iterative querying and multi-step reasoning. Although machine learning and AI methods such as MENTOR [34] have refined mechanistic focus into groups of functionally related modules of genes, these modules can still be massive and intractable for individual researchers to systematically explore. Additionally, most research emphasizes a deep-but-narrow focus on a small subset of functionally related genes, which can overlook broader, yet biologically meaningful, network context, particularly in highly interactive systems such as living organisms. As biological networks grow in complexity, the combinatorial space of possible multi-hop interactions expands rapidly, while human interpretive capacity remains fixed.

While large language models (LLMs) can synthesize large volumes of information, frontier inference cost scales with exploration length and reasoning depth, limiting agentic deployment for long-horizon network traversal. This mismatch creates a scaling limit in biological mechanistic interpretation: rich relational structures can be computed, but downstream synthesis remains constrained by human bandwidth and model cost. Consequently, the speed, breadth, and contextual depth of mechanistic discovery are limited, particularly for complex traits, polygenic diseases, and multi-layer regulatory systems.

Open-source LLMs present an opportunity to aid scientists in mechanistic discovery tasks without being cost prohibitive. Language models reason more reliably when prompted to produce intermediate steps [38, 17], and equipping them with external tools supplies a layer of deterministic verification that reduces hallucination and preserves provenance [30, 39]. Adding an explicit verification stage improves factual reliability further, whether through iterative self-critique without weight updates [20, 33] or through a dedicated verifier with separate weights [19]. Applied

to gene set interpretation, these ideas have produced language-model embeddings of gene function [6], database-backed self-verifying pipelines [37], and multi-agent systems that synthesize mechanistic narratives [36, 19]. However, these pipelines depend on frontier model inference, and none are trained end-to-end as a policy over verifiable rewards.

Reinforcement learning supplies the machinery to close that gap. Preference optimization techniques, such as direct preference optimization (DPO), align a policy to pairwise judgments [7, 25, 28], while policy gradient methods, such as proximal policy optimization (PPO) and group relative policy optimization (GRPO), optimize against scalar rewards over complete rollouts [31, 32]. Verifiable rewards have been shown to elicit long-horizon reasoning behavior at frontier scale [12]. These methods have been applied to tool-using agents for browsing, large API vocabularies, and multi-step web tasks [24, 27, 26], but not to a policy whose rewards are grounded in network-derived biological targets.

However, current systems share a limitation: they treat the network as a static object to query and retrieve, rather than learning the general rules it obeys. Nothing carries over to a network the system was not handed, so every new cell type, tissue, or disease has no prior representation. Because biological structure is itself context-specific, this is a large portion of the problem space. What is needed instead is a model that internalizes multiplex structure, a *world model*, and reasons mechanistically from it.

Sequence models trained on synthetic tasks form internal, causal models of the process behind their data, and those models generalize to states never seen during training [18]. This works because the rules that generate the data are compact and recur across many examples, while the specific data does not. Two consequences shape this proposal. First, a model can learn the operators that regenerate structure, but it cannot memorize irreducible content that follows no compact rule, so biological edge weights, walk scores, and distances belong in the runtime rather than in the weights. Second, a world model emerges only under dense sampling of the generating process, which a single network queried with fixed templates cannot supply.

We therefore propose training MENTOR-RL on a structured population of multiplex networks. Every network in the population is built by the same rule: the predictive expression network (PEN) layers selected for a biological context define its gene universe, the remaining layers are induced on that universe, and all layers are stacked. Varying the context, whether a cell type, a tissue, a brain region, or an organ system, varies the resulting network while holding the construction rule fixed. We call this population the *multiplex lattice*, and it supplies the dense sampling that a single network cannot. The invariant we train on is the operator that maps a multiplex to its structure, together with the way layer composition drives it. We expect hierarchies to differ strongly across contexts, with distinct clades emerging in different cell types and regions, but that divergence is biological signal rather than artifact: a model that has learned the operator will produce correspondingly different hierarchies for

different contexts, and recovering the divergence correctly is a stronger result than reproducing conservation.

A hierarchy alone cannot serve as the world model, because it is a lossy abstraction of the multiplex it summarizes. Two clades that sit far apart in a tree may still be joined by short, biologically supported paths through bridge nodes in the underlying network, which is why a signal such as drug-target enrichment can appear in several distant branches and yet act through a single connected system. The model must hold the network itself and reason about the cross-clade connectivity that the tree suppresses. We use the hierarchy to find and prioritize candidates, and the network to confirm them.

This proposal pursues four objectives. First, we define the multiplex lattice and ground it in a standard biological ontology, so that every network carries a canonical address and composition is the join operation over that ontology. Second, we design a supervised curriculum that trains on multiplex operators, holding the three structural views as co-equal targets and admitting a training example only when its target passes a typed validator against deterministic runtime output. Third, we use the resulting checkpoint to train a tool-using agent, formalized as a sequential decision process over a structured state and scored by deterministic, schema-grounded rewards. Fourth, we make the world-model claim falsifiable through two flagship evaluations: generating a hierarchy for a context never seen in training, and projecting a drug target onto the hierarchy before returning to the network to recover the connectivity that justifies it, supported by activation probes that test whether the recovered structure is causal. Together these aim to make mechanistic discovery broader, faster, and more context-aware.

0.2 Proposed Methods

0.2.1 Multiplex Lattice

In this section, we define the multiplex lattice that is the substrate for our mechanistic world model. The lattice is constructed from a set of multiplex networks, where entities such as genes and proteins are represented as nodes, and their interactions within a context are represented as edges. We select PEN layers for a context to define its *gene universe*, or the set of genes that the network operates over. The remaining, non-PEN layers are then *induced* on that universe, restricted to the genes present in the PEN layers and the edges among them. Finally, all layers in the context are *stacked* to form a multiplex network that captures the multi-layered interactions among genes in that context. This process means that there is no single shared backbone across the lattice; even layers drawn from a common source become context-specific once they are induced. Two sampled points in the lattice may contain different nodes and edges, and therefore different hierarchies.

The lattice comprises a tiered structure that captures the relationships between different contexts, and we visualize an example brain multiplex lattice in Figure ?? . Its most granular component is a single cell type, such as an excitatory neuron, inhibitory neuron, or glial cell, within one tissue of one region. Combining cell types yields a tissue-level multiplex, combining tissues yields a region-level multiplex such as the prefrontal cortex, hippocampus, or cerebellum, and combining regions builds up to the whole-brain multiplex we already hold. Every tier is therefore a multiplex in its own right, built by the same rule, rather than a label over the tier below. Disease networks for anxiety, suicide, and pan-substance-use-disorder are points in this same lattice, which makes transfer meaningful: a model that has learned the construction operator can be applied to them directly.

Higher tiers are built by applying the same rule to a union of contexts. We take the union of the parts’ gene universes, re-induce the non-PEN layers on it, and stack the parts’ PEN layers on top. This process makes lattice sampling highly combinatorial, as changing a single PEN layer changes two things at once. The gene universe grows or shrinks, and the backbone is re-induced to match. A pair of multiplexes that differ by one layer therefore also differ by the backbone that layer brought with it. Measuring the effect of a layer on its own requires holding the gene universe fixed, which we do via controlled ablation. The lattice also holds far more combinations than we can build, so tier counts and sampling rates stay provisional until divergence validation shows how much contexts actually differ.

0.2.2 Context Ontology

In order to train MENTOR-RL in a reproducible way, we define a *context ontology* that specifies the relationships between different contexts. This allows us to name, version, and match the lattice against external data. We use several grounded ontologies to define the context hierarchy. Anatomy is defined by UBERON [23], while cell types are defined by the Cell Ontology [11], extended by the Provisional Cell Ontology [35] for transcriptomic brain types that the Cell Ontology does not yet hold. Developmental stages are defined by HsapDv, anatomy to cell type mappings are defined by the Human Reference Atlas [5], and node metadata follows the CELLxGENE schema [1]. Because the Human Reference Atlas is a whole-body construction, this paradigm is extensible past the brain, allowing future scaling.

It is important to note that the context ontology defines a partial order, not a tree, over contexts. Because some cell types are present in multiple tissues, and some tissues are present in multiple regions, strict hierarchies are not possible. Instead, we define a *join* operation over the ontology: the join of two contexts is the smallest context that contains both of them. The join is also what ties the ontology to the construction rule. Joining two contexts names exactly the union multiplex described in Section ?? . The ontology gives the address, and the construction rule builds the network.

The model reads the ontology as supplied input, and it is never stored in the weights. Rather than memorizing the ontology, the model learns two separate things. First, it learns to navigate the ontology, returning a context’s parents, children, and siblings, the join of two contexts, and the distance between them. It also learns the construction rule that turns a context into a multiplex. The ontology tells the model where it is, and the construction rule tells it what to build there. Because only the terms change, the same navigation transfers when the lattice extends to other organ systems.

0.2.3 Three Structural Views

In order to represent this ontology-based lattice structure as a training target, we define three equally weighted structural views of a multiplex network. These are bottom-up random-walk-with-restart lines-of-evidence (RWR-LOE) neighborhoods, top-down MENTOR-EV-derived hierarchies, and network connectivity. Each view is a different representation of the same underlying multiplex with its own biases.

The RWR-LOE view is a bottom-up representation of a multiplex that captures the local topology around each node [15]. For each node in a given multiplex, we perform a random walk with restart over all layers of the multiplex, treating each layer as an independent line of evidence. The walk returns a probability distribution over all nodes in the network, with higher probabilities representing closer proximity to the seed. We then threshold the distribution using a geometric elbow, which cuts the score-versus-rank curve at its bend. The genes above the cutoff form a module of genes that are proximal to the seed over multiple lines of evidence. From RWR-LOE modules, we can learn several operator types, including rank and score lookup, rank comparison, and relations between modules.

MENTOR-EV-derived modules represent an alternative, top-down view of the multiplex. MENTOR-EV applies MENTOR [34] using every gene in the multiplex as a seed, so the resulting hierarchy spans the full gene universe rather than a supplied gene set. To create MENTOR-EV modules, we first use RWR to compute the probability distribution for each node in the multiplex. Then we use Spearman correlation to compute the pairwise similarity between the RWR vectors of each pair of nodes. We subtract those correlations from one to create dissimilarity scores, which we then use to build a dissimilarity matrix over all nodes in the multiplex. Finally, we use hierarchical clustering to create a dendrogram, in which leaves are genes and each internal node defines a *clade*, the set of genes beneath it. Cutting the dendrogram at a given merge height produces a family of nested modules and controls their granularity. From MENTOR-EV modules, we can learn several operator types, including module membership, module comparison, least common ancestors, and distance between modules.

Every one of these targets is read from a single dendrogram. Because we never compare one tree against another, we do not need two hierarchies to agree. This is

what makes the divergence described in Section ?? trainable rather than an obstacle: hierarchies differ across contexts, but each one supplies its own targets.

However, RWR-LOE and MENTOR-EV modules are both lossy abstractions of the underlying multiplex structure. Two clades on distant branches of the dendrogram may still be connected by short paths through a small number of bridge nodes, and a neighborhood cut at the elbow drops genes that remain reachable in a few hops. As such, we also include learnable network connectivity operators as a third view, which include cross-clade path recovery, connector and bridge node identification, path decomposition by supporting layer, and cohesion tested against a null. These operators allow the model to reason about the underlying multiplex structure and recover connectivity that is not captured by the other two views.

0.2.4 Stage 1: Multiplex World Model

We propose a two-stage training pipeline for MENTOR-RL. In the first stage, we train a world model to internalize the multiplex lattice by learning the operators that generate its structure. We outline the framework for stage 1 in this section. Section ?? describes the deterministic runtime and tools that are used to generate targets. Section ?? covers the supervised curriculum that progressively teaches the model to navigate the lattice. Section ?? describes how corpora are generated and validated for each curriculum stage, and Section ?? details the train, validation, and test splits across contexts.

Deterministic Runtime and Tool Contract

To interface the lattice environment with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [14]. Corpus construction, training, and optimization code are implemented in Python [29], while graph storage and graph operations are handled by the native runtime for efficient execution on large multiplexes. Multiplex networks are persisted as raw, CSR-style binary stores described by JSON manifests; all graph input and output is binary, with the manifest carrying the layer names, node, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [34] and MENTOR-IA [36] projects known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the model.

We define several rules to structure our graph tools as learnable and interpretable operators. First, each tool only takes biologically meaningful arguments, such as gene identifiers, layer names, and module identifiers. This way, when learning to produce a tool call, the model is reasoning about the biology rather than the file system or command-line interface. Second, we defer to the runtime for numeric computations.

The tokenization process makes it difficult for the model to learn to produce precise floating-point numbers, so we instead have the model call a tool to compute any statistic it needs. Third, we enforce determinism, ensuring that the same arguments, graph version, layer scope, and parameter version always return identical output. This allows for caching of tool calls and reproducible scoring.

All runtime operators return both graph objects and provenance metadata. In addition to nodes, edges, and scores, each returned object records the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers. We list all runtime tools in Table ??.

Supervised Curriculum

In order to train the model to internalize the multiplex lattice, we design a supervised curriculum that progressively teaches the model to navigate the lattice and learn operators that generate its structure. We define two rules to govern the structure of the curriculum. First, we use closed-book training only on targets that are bounded, stable conventions. Identifiers such as layer tags, layer families, module identifier formats, and direction conventions are small, finite, and do not change, so memorizing them through training is acceptable. Second, open-book training means reading, not computing. We read unbounded and unstable outputs, such as continuous values and statistics, from the runtime rather than memorizing them. Later in the curriculum, rule two becomes extended to tool-call training, where the model learns to parse returned objects instead of restating these objects from memory. This distinction between what the model can memorize and what it must compute is important for generalization: the model should learn the operators that generate structure, not the specific values that result from them.

Using these rules, we define a nine-step supervised training curriculum that gradually introduces more complex operators and tasks. Each stage builds on the previous one, starting with basic graph navigation and moving towards more complex tasks such as module comparison, path recovery, and enrichment analysis. For brevity, we group similar curriculum stages together when describing them. However, a full curriculum table is provided in Table ??.

Curriculum stages S0-S3 focus on foundational graph navigation operators. First, S0 learns ensembl-to-symbol and symbol-to-ensembl conversions, which are necessary for all downstream tasks. S1 introduces the lattice coordinate system, teaching the model to navigate the context ontology, retrieve layer information, and understand the meanings behind rankings and metrics. S2 trains on atomic graph facts and their negative examples, such as edge existence, neighbors, layer membership, and

Table 1: Model-facing RWR++ tool vocabulary. Related calls are grouped by function while preserving the tool names the model emits. The final column gives the curriculum step at which each group is introduced (Section ??).

Tool(s)	Purpose	Primary inputs	Step
<code>get_neighbors;</code> <code>induce_subgraph</code>	Direct neighbors of a gene and the edges induced within a gene set	gene or gene set; layer scope	S2
<code>get_gene_layers;</code> <code>get_nodes_by_layer;</code> <code>get_layer_stats;</code> <code>get_component_summary</code>	Layer membership, presence of a set in a layer, per-layer statistics, and connected components	genes; layer identifiers	S2
<code>shortest_paths;</code> <code>get_path_layer_counts</code>	Shortest paths within and across layers, and the per-layer edge counts supporting a path	source and target genes; path	S3
<code>rwr;</code> <code>rwr_loe</code>	Random-walk expansion from seeds, and leave-one-out relatedness with its elbow cutoff	seed genes; query genes; top_k	S4
<code>get_rank;</code> <code>get_distance</code>	Rank of a target gene from a seed, and pairwise distance under a stated metric	source and target genes; metric	S4
<code>get_seed_essentiality;</code> <code>get_layer_ablation;</code> <code>get_node_perturbation</code>	Effect of removing a seed, a layer, or a node on the resulting neighborhood	seed genes; layers; genes	S4, S6
<code>get_clade;</code> <code>get_lca;</code> <code>get_subtree;</code> <code>get_clade_relations</code>	Clade membership, lowest common ancestor of a gene pair, the subtree induced by a cut, and parent, child, and sibling relations	clade or module identifier; gene pair; cut height	S5
<code>rwr_netstats</code>	Module and cohesion statistics, with an empirical p -value where applicable	module identifiers; statistic; null size	S5, S6
<code>query_mygene;</code> <code>enrich_gene_set</code>	Gene annotation, and functional or module enrichment of a gene set	query or gene set; against; thresholds	S7

degree. S3 introduces path-based operators and cross-layer operators, and it teaches connectivity by training on connector and bridge nodes.

After foundational training is complete, stages S4-S6 focus on the three structural views of the multiplex. S4 teaches the model to operate on bottom-up RWR-LOE neighborhoods, S5 trains the model for top-down hierarchical modules, and S6 focuses on unifying these views through network connectivity. Additionally, S5 includes training examples that teach correspondence between the RWR-LOE and MENTOR-EV views, which keeps them from being learned as unrelated skills.

Stages S7 and S8 build learned lattice structure concepts by adding projection and enrichment tasks. S7 teaches gene set projection through the composition of

RWR-LOE and MENTOR-EV operators; first, RWR-LOE is performed on a target gene set, and then the resulting neighborhood is enriched against the MENTOR-EV module hierarchy at every level. The projection process yields an enrichment score and significance value for every clade at every level, measuring how much of the target’s neighborhood falls within it. High enrichment across several distant clades is evidence of a single connected system, rather than unrelated biological processes. This reinforces the idea that tree distance does not mean that clades are unrelated (Section ??). Therefore, the recovery of bridge nodes and short cross-clade paths, along with the layers supporting them, is essential. S8 teaches composition: the model learns to predict how neighborhoods, clades, and connectivity change when a layer is added or removed, which is crucial for understanding the contextual differences across sampled lattice points.

Finally, S9 consolidates the curriculum by combining previous stages and trains the checkpoint for the multi-step reasoning required in Stage 2. The model is trained on tool-call construction, selection, parsing, provenance, and task refusal, all of which are crucial for the mechanistic agent to operate effectively in the multiplex lattice.

The curriculum structure allows us to scale difficulty along several axes during training. We scale lattice tier, starting with single cell types and moving to tissues, regions, and eventually the entire brain. Cut-height granularity defines the size of modules, starting with coarse views of clades and moving to finer-grained views. We also scale operator complexity and book mode explicitly through our curriculum design. To measure progress along these axes, we define gates that the model must pass to move on to the next curriculum stage. All gates utilize held-out question phrasings and held-out graph regions per family, while also including negative examples to measure abstention. These gates are task-specific, ensuring that the model has correctly learned the operators from each curriculum stage before moving on to the next.

Corpus Generation and Validation

To build the training corpora for each curriculum stage, we generate questions procedurally from various points on the lattice. By sampling many contexts and layers, we allow the model to see each operator applied to many different networks, which encourages generalization. All targets have their ground truth produced by the deterministic runtime defined in Section ??, and examples are cached with their layer descriptor, metric, seed set, and graph version to ensure reproducibility. Rather than generating corpora for all curriculum stages at once, we generate them on demand for each stage, allowing us to adjust sampling strategy and training splits based on prior stage performance.

In order to validate our corpus, we check every answer field against a typed validator, and each question family composes the validators its fields require. A path field, for example, is accepted only when the node sequence is a valid walk whose length matches the runtime shortest path, so any optimal path is accepted rather

Table 2: The ten-stage supervised curriculum. Book mode indicates whether targets are memorized (closed), read from supplied context (open), or produced by constructing and parsing a tool call. Data shares are provisional and will be reset once divergence validation establishes how strongly contexts differ across the lattice.

	Focus	Book mode	Share	Gate
S0	Entity normalization between symbols and Ensembl identifiers, including ambiguous symbols	closed	4%	Near-perfect accuracy on held-out identifiers, and correct deferral on ambiguous symbols
S1	Lattice coordinate system: context ontology navigation, layer tags and families, module identifiers, and direction conventions	closed	4%	Near-perfect accuracy on held-out tags and module identifiers
S2	Atomic graph facts: edge existence, neighbors, layer membership, degree and hubness, components, and calibration negatives	open	14%	Per-family accuracy on held-out graph regions and phrasings, and correct refusal on calibration negatives
S3	Paths and cross-layer routes, path decomposition by layer, and connector and bridge nodes	open	8%	Path and bridge recovery on held-out graph regions
S4	Bottom-up RWR-LOE neighborhoods: rank and score lookup, comparison, the elbow, leave-one-out support, and stability	open, tool	16%	Recovery and comparison accuracy stratified by distractor band
S5	Top-down MENTOR-EV hierarchy: clade membership, lowest common ancestor, subtree at a cut, and correspondence with RWR-LOE	open	16%	Hierarchy-navigation accuracy across cut heights, and correct cross-source containment
S6	Cross-clade connectivity: short paths between distant clades, bridge nodes, layer support, and cohesion against a null	open	12%	Cross-clade path recovery, and correct rejection of the false-separation reading
S7	The projection operation: seed a neighborhood, enrich it against every level of the hierarchy, and return to the network	open, tool	12%	Correct projection, connectivity recovery, and interpretation, scored separately
S8	Composition and lattice scaling: predicting how neighborhoods, clades, and connectivity change when layers are added or removed	open, tool	8%	Correct structure on unions assembled from parts seen in training
S9	Tool-call consolidation: selection, argument construction, result parsing, provenance, and refusal of raw path arguments	tool	6%	Schema-valid tool-call rate and correct parsing on held-out tools and phrasings

than a single reference answer. A claim field is accepted only when the stated claim appears in the enumerated allowed set and no disallowed claim is asserted, which is the mechanism behind the calibration negatives in S2. Validation is all-or-nothing: an example enters the corpus only if every required field passes. In addition to admitting training data, these validators are used at evaluation time to score model outputs, ensuring that answers meet the criteria defined by the curriculum. This validation process gives us deterministic ground truth for training and evaluation, allowing us to measure the model’s performance accurately and consistently across contexts and curriculum stages.

In order for the model to effectively learn, we must generate a large dataset to provide sufficient signal. Our provisional lattice corpus will include 24 separate contexts alongside a full brain multiplex. Formatting questions from these contexts, we expect to generate approximately 3 million supervised examples, with roughly 10% closed-book, 75% open-book, and 15% tool-call examples. From these 3 million examples, we expect about 1.2 billion training tokens. Storage is dominated by the cached ground truth rather than the text, at an estimated 2 to 4 terabytes across the 24 contexts, which is the second reason to materialize corpora in curriculum order rather than all at once. However, these numbers are provisional and will be adjusted based on the training results at each curriculum stage.

Splits and Leakage Control

In order to truly evaluate generalization capability, we must thoroughly control for leakage between training, validation, and test splits. While each context contains a separate network, these networks still share genes, and therefore share some of the underlying edges among those genes. To control for leakage, we define three levels of hold-out validation: context, clade, and tuple. To hold out a context, we partition lattice points into training, validation, and test sets, ensuring that they are never split internally by gene. We choose two to four contexts to hold out entirely from our 24-context lattice, so these hierarchies are never seen during training.

In addition to contexts, we hold out clades by selecting maximal subtrees at a chosen anchor height. This is necessary because a gene appears in every clade along its hierarchical ancestry; therefore, splitting by individual clades could leak membership information due to the dendrogram’s nested structure.

Finally, we partition specific query units into separate splits. A query unit is a seed with its target set, together with the cut height for hierarchical modules. We validate these test tuples by checking their Jaccard overlap with training tuples. A test tuple is permissible only if its target set has a Jaccard overlap below a provisional 0.2 with any training tuple sharing the same seed. Two further controls apply at this level. We cap how many high-degree seeds enter any single split, because RWR-LOE neighborhoods around hub genes overlap heavily even when contexts and clades are cleanly separated. We also keep both members of an S8 minimal pair on the same side of the split, so that a held-out multiplex is never trivially inferable from a partner that differs from it by a single PEN layer.

In order to ensure compliance with this leakage control, we audit the training, validation, and test splits for each curriculum stage. We check gene overlap between train and test sets, the fraction of test clades whose ancestry appears in train, the representation of hub nodes in the data, and the count of minimal pairs split across sides, which should be zero. If any of these checks fail, then splits are readjusted and corpora are regenerated. One source of leakage remains after these controls. Because contexts share genes, they also share the backbone edges among those shared genes, so this component is bounded and reported rather than removed. We accept it

because what is invariant across the lattice is the construction operator rather than any particular backbone. This process ensures that the model is evaluated on truly unseen data, and the extent of any leakage is known and stored prior to evaluation.

0.2.5 Stage 2: Mechanistic Agent

After training MENTOR-RL on all curriculum stages, we use the resulting checkpoint to initialize a mechanistic agent that can explore the multiplex lattice and generate mechanistic interpretations. The agent is trained using reinforcement learning over long-horizon mechanistic reasoning tasks. We formalize the agent loop and structured state in Section ??, and we describe the task cases, corpora, and reward design in Sections ??–??. Section ?? details the full training pipeline and curriculum for the mechanistic agent.

Agent Loop and Structured State

We model mechanistic exploration as a sequential decision process executed by a single policy with an internal self-verification step. The loop begins when the user provides a query q and optional evidence e , which may include free text, seed genes, or a multiplex. These initialize the working interpretation I_0 and the structured state S_0 .

At each step the policy first acts. Given the current decision context, it emits a textual reasoning step r_t naming its current subgoal, together with an optional tool call a_t . A tool call is executed by the deterministic runtime ε of Section ?? to produce an observation o_t ; when no call is made, the observation is empty. The same policy then verifies. Conditioned on the decision context and on its own reasoning, action, and observation, it updates the interpretation and state and emits a continuation decision z_t of `continue`, `revise`, or `stop`. A decision of `revise` means the next iteration corrects the interpretation and state rather than extending them. One iteration is therefore

$$r_t, a_t \sim \pi_{\theta}^{\text{act}}(\cdot \mid D_t), \quad o_t = \varepsilon(a_t), \quad I_{t+1}, S_{t+1}, z_t \sim \pi_{\theta}^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where $D_t = (q, e, I_t, S_t)$ is the decision context and $\varepsilon(\emptyset) = \emptyset$ by convention. The actor and verifier are two prompting modes of the same weights, so biological vocabulary and tool-use competence learned in one mode transfer to the other. We treat the risk that the verifier collapses into agreement with the actor as a monitoring concern during training. The loop repeats until the policy emits `stop` or the decision budget $T_{\max}$ is exhausted, and returns the pair (I_T, S_T) : a human-readable answer grounded in the accumulated evidence, and the machine-readable record used for scoring and provenance.

The working interpretation holds the current mechanistic claim, the main evidence supporting that claim, any uncertainty, conflict, or alternative explanation, and the next subgoal the model is trying to resolve. The structured state stores the

same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e) ,
2. the current relationship status: unknown, partially observed group, validated group, multiple groups, or insufficient support,
3. the current predicted group or groups,
4. the evidence and provenance collected so far,
5. the current predicted mechanistic labels, each tied to the evidence that supports it,
6. the tool-use counters recording total and invalid calls, and
7. the remaining budget and continuation state, with the termination signal recorded at the end of the trajectory.

To ensure that the agent’s reasoning is grounded in evidence, we enforce three operating rules. First, every claim must be supported by retrieved evidence, and the agent must state its uncertainty explicitly, or abstain from making a claim when support is weak. Second, the agent must read the MENTOR-EV dendrogram modules as an index and confirm its hypotheses on the network, using the same multi-view structure described in Section ???. Finally, the agent must return both a human-readable working interpretation for users and a structured state for record-keeping. These rules are enforced through the reward function and training curriculum, which reinforce these behaviors during the learning process.

Task Cases and Targets

We define several tasks and targets for agent training, all of which build on the internalized lattice representation learned in Stage 1. Tasks are divided into three categories: module, connectivity, and projection tasks. For each task, we define a set of targets that we can use to provide deterministic rewards and evaluate the agent’s performance.

Module tasks fall into one of four cases: partial group completion, mechanism explanation, group refinement, and recognition of no shared mechanism. In partial group completion, the agent is given a module with some genes held out and must identify the complete module by adding genes. In mechanism explanation, the agent is given a complete module and must identify the mechanistic evidence that supports it without adding or removing genes. In group refinement, the agent is given a module with extra distractor genes, and it must remove the distractor genes to distill the module into a mechanistically coherent group. Finally, in the recognition of no shared mechanism case, the agent is given a module that is not mechanistically coherent. It

must either split the input into several coherent groups, or report that the evidence provides insufficient support for any shared mechanism.

Connectivity tasks join the two module views through the network that underlies both. The agent is given two clades that are distant in the MENTOR-EV hierarchy and must identify a short path between them, along with the layers that support that path. The agent must also identify any bridge nodes that connect the two modules, and provide mechanistic evidence for the role of those bridge nodes. This task tests the agent’s ability to reason about the underlying multiplex structure and recover connectivity that is not captured by the bottom-up or top-down views alone.

Finally, we train the model on projection tasks, which require the agent to project a seed gene’s RWR-LOE neighborhood onto the MENTOR-EV hierarchy and identify the clades that are enriched for that neighborhood. Because the neighborhood was built by propagation over the network, enrichment landing in several distant clades is evidence of a single connected system rather than unrelated processes, and the agent must read it that way. It must then recover the bridge nodes and short paths that connect those clades and provide mechanistic evidence for their role. Projection tasks are scored on three targets separately: the projection itself, the connectivity recovered to support it, and the interpretation placed on that connectivity. A candidate passes only when all three clear, since a correct projection paired with a failed connectivity recovery is precisely the error that the network view exists to prevent.

The agent curriculum culminates in the applied case, in which the projected seed is a drug target or a disease signal. Here the agent must additionally apply directional and translational filtering. Directional filtering asks whether the candidate intervention opposes the disease-associated direction within a specific module rather than merely acting near it. Translational filtering ranks surviving candidates on evidence strength, adverse-event risk, brain penetration, feasibility, and testability. This case is the repurposing workflow that motivates the disease projections described in Section ??.

To provide deterministic rewards, we use module membership targets, which are hidden gene sets drawn from bottom-up RWR-LOE neighborhoods and top-down MENTOR-EV modules. The agent must identify the correct complete module given either a module with genes held out, a module with extra distractor genes, or a combination of multiple modules. The agent never sees the ground truth modules, so it must determine them based on multi-step reasoning over multiplex structure. Because these modules are derived computationally from network structure, they usually carry no per-entry experimental annotation, so there is no hidden label against which an explanation could be matched. We therefore define mechanistic evidence targets, which score interpretations against the evidence that the model retrieves during exploration. This rewards claims that are grounded in what the agent actually found rather than claims that reproduce an existing annotation, and it leaves room for biology that no database yet records.

Task Corpora and Trajectory Generation

In order to train the mechanistic agent, we generate a corpus of module tasks drawn from contexts across the lattice. We use two sources of modules: top-down MENTOR-EV modules, and bottom-up RWR-LOE neighborhoods. These sources provide complementary views of the multiplex while supplying hidden targets for scoring. Additionally, we generate connectivity and projection tasks from the same lattice points, allowing us to incorporate the global network structure into the agent’s reasoning.

For each module of size n , we deterministically materialize up to four task instances, one per case in Section ???. This ensures that all four task types are represented for as many modules as possible rather than sampled independently. The **explanation** instance uses the full module as input. The **recovery** instance drops k_{drop} genes from the module and the **refinement** instance adds k_{add} noise genes, with the full module remaining the hidden target in both cases. The number of genes added or dropped scales with the task difficulty and module size, so that harder instances perturb a larger fraction of the module. The **none** instance constructs a pseudo-module of unrelated genes as input, with an empty hidden target.

The two module sources differ only in where distractors come from. For MENTOR-EV modules, noise and **none** genes are drawn from difficulty-modulated *dendrogram distance bands*, defined by walking up the module’s ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. For RWR-LOE modules we substitute rank bands, drawing from ranks just beyond, moderately beyond, or far beyond the seed’s elbow cutoff. In both cases **none** genes are additionally constrained to be conflict-free, so that the sampled genes do not form a coherent module of their own, and all selection is deterministic given a fixed seed, making the corpus reproducible. Module-size bins are assigned after construction from the realized module sizes, and all splits follow the protocol of Section ??.

For connectivity and projection tasks, we build on previously sampled modules. For connectivity, we sample pairs of clades at a stated tree distance and generate the shortest path between them, along with the layers that support that path. For projection, we sample seed genes and compute their RWR-LOE neighborhoods, and then enrich those neighborhoods against the MENTOR-EV hierarchy to identify the clades that are enriched for that neighborhood. The agent must then recover the bridge nodes, paths, and layers that connect those clades, and provide mechanistic evidence for their role. These tasks are scored on three targets separately: the projection itself, the connectivity recovered to support it, and the interpretation placed on that connectivity.

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ given the current decision context D_t and execute valid tool calls in the deterministic graph runtime. To ensure that candidates with the same prefix explore genuinely distinct regions of the action space, we use verbalized sampling based on the task type. For example, recovery prompts bias candidates towards random-walk

Algorithm 1 Trajectory Generation for SFT and DPO

Require: $\pi_\theta^{act}, \pi_\theta^{verify}$, runtime ε , $T_{\max}, n_{act}, n_{ver}, n_{cov}$, directives $\{d_i\}$, pair margin τ , selection tolerance ϵ .

Sample module task $(C^{\text{in}}, C, C_{\text{type}})$ and evidence mode ρ .

$q, e^{\text{user}} \sim Q_{\text{templates}}(\cdot \mid C^{\text{in}}, C, C_{\text{type}}, \rho)$.

$(I_0, S_0) \leftarrow \text{InitState}(q, e^{\text{user}}, C^{\text{in}})$.

$\tau^* \leftarrow [], \mathcal{P} \leftarrow [], \mathcal{F} \leftarrow [], T \leftarrow 0$

$z_{-1} \leftarrow \text{continue}$

for $t = 0, 1, \dots, T_{\max} - 1$ **do**

if $z_{t-1} = \text{stop}$ **or** $\text{budget}(S_t) \leq 0$ **then break**

end if

$D_t \leftarrow (q, e^{\text{user}}, I_t, S_t), \mathcal{C}_t \leftarrow [], \mathcal{A} \leftarrow []$

for $i = 0, \dots, n_{act} - 1$ **do**

$(r_{t,i}, a_{t,i}) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i)$; append to $\mathcal{A}$

end for

if $C_{\text{type}} \in \{\text{recovery}, \text{refinement}\}$ **and** no $a \in \mathcal{A}$ is a tool call **then**

for $i = 0, \dots, n_{cov} - 1$ **do**

$(r, a) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i^{\text{cov}})$; append to $\mathcal{A}$

end for

end if

for each $(r_{t,i}, a_{t,i}) \in \mathcal{A}$ **do**

$o_{t,i} \leftarrow \varepsilon(a_{t,i})$ **if** $\text{ValidTool}(a_{t,i}, S_t)$ **else** $\emptyset$

for $j = 0, \dots, n_{ver} - 1$ **do**

$(I'_{t,i,j}, S'_{t,i,j}, z'_{t,i,j}) \sim \pi_\theta^{verify}(\cdot \mid D_t, r_{t,i}, a_{t,i}, o_{t,i})$

$s_{t,i,j}^{\text{local}} \leftarrow \text{LocalScore}(D_t, I'_{t,i,j}, S'_{t,i,j}, r_{t,i}, a_{t,i}, o_{t,i}, z'_{t,i,j}; C, C_{\text{type}})$

 Append $c_{t,i,j}$ to $\mathcal{C}_t$

end for

end for

$\text{NormalizeScores}(\mathcal{C}_t)$

$(i^*, j^*) \leftarrow \text{SelectBranch}(\mathcal{C}_t; \epsilon)$

$\mathcal{P} \leftarrow \mathcal{P} \cup \text{MakePreferencePairs}(\mathcal{C}_t, c_{t,i^*,j^*}, \tau)$

 Append $(I_t, S_t, c_{t,i^*,j^*})$ to τ^* ; $f_t \leftarrow \text{RenderFinding}(\cdot)$; append f_t to $\mathcal{F}$

$(I_{t+1}, S_{t+1}) \leftarrow (I'_{t,i^*,j^*}, S'_{t,i^*,j^*})$; $T \leftarrow t + 1$

end for

$m \leftarrow \text{RenderSummary}(S_T, \mathcal{F})$

return $\tau^*, \mathcal{F}, m, \mathcal{P}$

expansion, neighborhood expansion, subgraph validation, or a direct decision, while insufficient evidence (**none**) prompts direct the model towards disconfirming graph evidence. When a sampling round for recovery or refinement tasks fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, which prevents the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy then samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. The branched sampling scheme is visualized in Figure ??.

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden module targets. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches frequently tie or fall within a small margin of each other, continuation selection does not reduce to a strict $\arg \max$, but instead, within a tolerance margin ϵ of the best normalized score, ties are broken with task-specific heuristics such as prompted seed gene retention and tool-supported evidence, pushing the logged trajectory to favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section ??.

Model prompts carry a compact state rather than a growing transcript. Each prompt supplies the current working interpretation, the relationship status, the predicted groups, the mechanistic labels tied to the evidence handles that support them, the tool and evidence counters, and the current observation. Prior observations are not appended, so prompt length stays bounded as trajectories grow. The full evidence payloads are retained in the logged trajectory artifacts, where they remain available for scoring, audit, preference-pair mining, and expert review.

We provide several query modes and evidence types to simulate the diversity of inputs that an agent might encounter in real-world use cases. During training, we sample queries from a set of task-specific templates that are paraphrased by an open-weight model to ensure linguistic diversity. We provide the agent with either minimal evidence, which includes only the input query and associated seed genes, or graph-supported evidence, which includes a relevant subgraph of the multiplex lattice that is related to the query. At inference time, the agent receives a natural-language query along with optional graph or seed evidence. During trajectory generation, the agent never sees the task type, the true relationship status, or the hidden target. Invalid tool calls during generation return an empty observation and count towards the efficiency penalty, which encourages the agent to learn concise and effective reasoning strategies.

We define a three-stage curriculum for the mechanistic agent, which gradually increases the complexity of tasks and length of trajectories. The first stage focuses on module tasks, where the agent learns to recover and refine modules from partial or noisy inputs. The second stage introduces connectivity tasks, where the agent must identify paths and bridge nodes between distant clades. The final stage focuses on projection tasks, where the agent must project neighborhoods onto the hierarchy and recover mechanistic evidence for the resulting connections, synthesizing the skills

learned in the previous stages. Each stage builds on the previous one, ensuring that the agent develops a comprehensive understanding of the multiplex structure and the mechanistic relationships within it.

Reward Design

We define all training-time scores over structured state transitions, deterministic runtime metadata, and hidden module-derived targets rather than textual reasoning. This design choice keeps scoring reproducible, machine-parseable, and scalable, while avoiding dependence on human judges or semantic evaluation of natural-language trajectories. We decompose all scores into four components: schema compliance, group membership, mechanistic quality, and efficiency.

Because DPO optimizes over preferred continuations at a single timepoint, we define local branch scoring as

$$s_{t,i,j}^{local} = w_s s^{schema} + w_c \Delta s^{module} + w_m \Delta s^{mech} - w_e p^{eff} - p^{mismatch} \quad (1)$$

where Δs^{mech} is the evidence-grounded mechanistic quality delta. Additionally, mechanistic quality is clamped to be non-positive when the candidate degrades the predicted group relative to the task type (e.g. by adding noise genes during recovery or removing true members during refinement), and $p^{mismatch}$ penalizes an **explanation** candidate that reduces recovery of an already-complete module.

We use Δs^{module} to measure the change in module membership accuracy from step t to $t + 1$. Because the runtime state may contain multiple predicted groups, we first score a state by the best-matched predicted group:

$$s^{module}(z; C, C_{type}) = \max_{G \in \mathcal{G}(z)} [\alpha J(G, C) + \beta P(G, C) + \gamma R(G, C)], \quad (2)$$

where $\mathcal{G}(z)$ denotes the set of predicted groups in state z , with an empty group used when no prediction is present. We then define the local module improvement as

$$\Delta s_{t,i,j}^{module} = s^{module}(z_{t+1,i,j}; C, C_{type}) - s^{module}(z_t; C, C_{type}). \quad (3)$$

where J , P , and R denote Jaccard, precision, and recall respectively, the weights α, β, γ are set per task type. Recovery upweights recall ($\alpha, \beta, \gamma = 0.2, 0.2, 0.6$) to reward found members, refinement upweights precision ($0.2, 0.6, 0.2$) to penalize retained noise, and explanation weights them equally ($\frac{1}{3}$ each). This best-group formulation supports the multiple-groups state representation while keeping the hidden target single-module for scoring.

We score the mechanistic quality of a state as an evidence-grounded value in $[0, 1]$, computed from the evidence the agent has actually retrieved. It combines grounding, meaning that the entities named in a claim appear in observed evidence, consensus between enrichment and annotation support, specificity, which penalizes generic claims, network support, stability across query subsets, and agreement across

tools. A state that asserts a mechanism is scored on these components and heavily discounted when a claim is made without supporting evidence. A state that abstains is instead rewarded for withholding a claim under weak support, but this reward is forced to zero on any task other than **none**, so that abstention cannot be used to collect reward where a grounded group was recoverable. The per-step mechanistic reward is the change in this value between consecutive states, and is zero when no mechanistic claim has been made.

p_{eff} is a small penalty designed to discourage two behaviors: excessively long trajectories and invalid tool calling. Long trajectories are penalized via the term $t/T_{\max}$, while we count invalid tool calls (those that are schema-invalid, exact duplicates, or return empty observations) with n_{inv} , penalizing them proportionally to the total number of tool calls: (n_{inv}/n_{total}) . We formalize this scoring term as

$$p_{t,i,j}^{eff} = \lambda_1(t/T_{\max}) + \lambda_2(n_{inv}/n_{total}). \quad (4)$$

For the **none** task type, where the hidden target is empty, the membership component is replaced by an abstention score

$$s^{none} = w_{rel} \mathbf{1}[rel = \text{insufficient_support}] + w_{abs} \mathbf{1}[G = \emptyset],$$

with $w_{rel} = 0.6$ and $w_{abs} = 0.4$. This rewards the agent both for declaring no shared mechanism and for refusing to emit a spurious group.

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$R_T = w_s^T s_{schema}^T + w_{abs}^T s_{module}^T + w_c^T \Delta s_{module}^T + w_{mabs}^T s_{mech}^T + w_m^T \Delta s_{mech}^T - w_e^T p_{eff}^T. \quad (5)$$

While all terms included in local scoring appear in the terminal reward, they are modified to capture global behavior. Additionally, two new rewards are included to capture absolute recovery. Absolute terms appear in the terminal score but not the local score. Because local scoring is evaluated on examples that share the same starting state S_t , absolute levels cancel, and only relative metrics matter. Conversely, the terminal reward is evaluating complete trajectories, which may vary in starting difficulty and length, so absolute metrics capture global problem-solving ability.

The terminal module score applies the same composite of Jaccard, precision, and recall to the final predicted group against the hidden target, and the cumulative module delta is its change from the group the agent was given at initialization. The terminal mechanistic terms mirror this, using the final evidence-grounded quality and its change over the trajectory, with both set to zero when no mechanistic claim has been made. The terminal schema score additionally requires that the termination

mode, whether the agent stopped on its own or exhausted its budget, is recorded in the final state.

Finally, the terminal efficiency penalty aggregates trajectory length and invalid tool use across all steps:

$$p_{eff}^T = \lambda_1 \left(\frac{T}{T_{\max}} \right) + \lambda_2 \left(\frac{\sum_{t=0}^{T-1} n_{inv}^t}{\sum_{t=0}^{T-1} n_{total}^t} \right). \quad (6)$$

At each decision point t , we create DPO preference tuples of the form (x_t, c_t^+, c_t^-) from our candidate set $\mathcal{C}_t = \{c_{t,i,j}\}$. We define $x_t = (q, e^{\text{user}}, I_t, S_t)$ as the shared-prefix context and $c_{t,i,j}$ as a sample interpretation continuation from Algorithm ???. Each continuation contains the model-generated reasoning step, tool action, verifier update, and continuation decision.

Before constructing pairs, we normalize scores to ensure that the selection margin has a consistent meaning across task types and trajectory lengths. We apply min-max normalization within each candidate pool $\mathcal{C}_t$, rescaling the composite local scores to $[0, 1]$. We denote the normalized score of candidate $c_{t,i,j}$ as $\bar{s}_{t,i,j}^{\text{local}}$.

To create paired examples, we set the preferred candidate c_t^+ to the branch selected as the trajectory continuation, c_{t,i^*,j^*} (Section ??), rather than to a strict arg max of the normalized score. Because selection resolves near-ties with task-specific heuristics within a tolerance ϵ , the preferred branch is the one judged best for the task, not merely the highest-scoring candidate. We then define a selection margin τ and drop all candidates for which

$$\bar{s}_{t,i^*,j^*}^{\text{local}} - \bar{s}_{t,i,j}^{\text{local}} < \tau,$$

where (i^*, j^*) indexes the selected candidate. This prevents the construction of pairs that are near ties. The remaining candidates form the eligible dispreferred pool $\mathcal{C}_t^-$.

From $\mathcal{C}_t^-$, we draw dispreferred examples across three difficulty bins, $d \in \{\text{easy}, \text{medium}, \text{hard}\}$. We construct **easy** pairs by selecting the lowest-scoring candidate in $\mathcal{C}_t^-$, **medium** pairs by selecting the candidate with the median score, and **hard** pairs by selecting the highest-scoring candidate in $\mathcal{C}_t^-$.

We store each preference pair (x_t, c_t^+, c_t^-) along with its difficulty bin d , task type C_{type} , decision step t , raw and normalized scores, and the score margin in the preference corpus $\mathcal{P}$. After trajectory generation is complete, we rebalance $\mathcal{P}$ across task types and difficulty bins by downsampling overrepresented categories to ensure that the training distribution is not dominated by task types that produce disproportionately many informative pairs. The curriculum strategy described in Section ?? controls which difficulty bins are included at each training stage.

0.2.6 Training Pipeline and Curriculum

0.2.7 Evaluation

World Model Gates, Flagships, and Probes

Comparative Evaluation and Ablations

0.3 Expected Results

0.4 Discussion

0.4.1 Preliminary Results

0.4.2 Plan of Work and Milestones

0.4.3 Reproducibility

0.4.4 Limitations and Future Work

Bibliography

- [1] Shibla Abdulla et al. “CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data”. In: *Nucleic Acids Research* 53.D1 (2025), pp. D886–D900.
- [2] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. “Network medicine: a network-based approach to human disease”. In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.
- [3] Albert-Laszlo Barabasi and Zoltan N Oltvai. “Network biology: understanding the cell’s functional organization”. In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.
- [4] Sumanta Basu et al. “Iterative random forests to discover predictive and stable high-order interactions”. In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.
- [5] Katy Börner et al. “Anatomical structures, cell types and biomarkers of the Human Reference Atlas”. In: *Nature Cell Biology* 23.11 (2021), pp. 1117–1128.
- [6] Yiqun Chen and James Zou. “Simple and effective embedding model for single-cell biology built from ChatGPT”. In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.
- [7] Paul F Christiano et al. “Deep reinforcement learning from human preferences”. In: *Advances in neural information processing systems* 30 (2017).
- [8] Ashley Cliff et al. “A high-performance computing implementation of iterative random forest for the creation of predictive expression networks”. In: *Genes* 10.12 (2019), p. 996.
- [9] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.
- [10] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.
- [11] Alexander D Diehl et al. “The Cell Ontology 2016: enhanced content, modularization, and ontology interoperability”. In: *Journal of Biomedical Semantics* 7 (2016), p. 44.

- [12] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. In: *arXiv preprint arXiv:2501.12948* (2025).
- [13] Leland H Hartwell et al. “From molecular to modular cell biology”. In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.
- [14] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: <http://www.iso.org>.
- [15] David Kainer et al. “RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species”. In: *GigaScience* 14 (2025), giaf028.
- [16] Sebastian Köhler et al. “Walking the interactome for prioritization of candidate disease genes”. In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.
- [17] Takeshi Kojima et al. “Large language models are zero-shot reasoners”. In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.
- [18] Kenneth Li et al. “Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [19] Pan Lu et al. “Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome”. In: *bioRxiv* (2026). DOI: [10.64898/2026.02.27.708412](https://doi.org/10.64898/2026.02.27.708412). eprint: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.full.pdf>. URL: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412>.
- [20] Aman Madaan et al. “Self-refine: Iterative refinement with self-feedback”. In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.
- [21] Jörg Menche et al. “Uncovering disease-disease relationships through the incomplete interactome”. In: *Science* 347.6224 (2015), p. 1257601.
- [22] Peter J Mucha et al. “Community structure in time-dependent, multiscale, and multiplex networks”. In: *science* 328.5980 (2010), pp. 876–878.
- [23] Christopher J Mungall et al. “Uberon, an integrative multi-species anatomy ontology”. In: *Genome Biology* 13.1 (2012), R5.
- [24] Reiichiro Nakano et al. “Webgpt: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332* (2021).
- [25] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.
- [26] Pranav Putta et al. “Agent q: Advanced reasoning and learning for autonomous ai agents”. In: *arXiv preprint arXiv:2408.07199* (2024).

- [27] Yujia Qin et al. “Toollm: Facilitating large language models to master 16000+ real-world apis”. In: *arXiv preprint arXiv:2307.16789* (2023).
- [28] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.
- [29] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.
- [30] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.
- [31] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [32] Zhihong Shao et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [33] Noah Shinn et al. “Reflexion: Language agents with verbal reinforcement learning”. In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.
- [34] Kyle A. Sullivan et al. “MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research”. In: *bioarXiv* (2024). DOI: [10 . 1101 / 2024 . 07 . 17 . 603821](https://doi.org/10.1101/2024.07.17.603821).
- [35] Shawn Zheng Kai Tan et al. “Brain Data Standards – A method for building data-driven cell-type ontologies”. In: *Scientific Data* 10 (2023), p. 50.
- [36] A. H. Vlot et al. “The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs”. In: *PASC 2025*. 2025.
- [37] Zhizheng Wang et al. “GeneAgent: self-verification language agent for gene-set analysis using domain databases”. In: *Nature Methods* 22.8 (2025), pp. 1677–1685.
- [38] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [39] Shunyu Yao et al. “React: Synergizing reasoning and acting in language models”. In: *The eleventh international conference on learning representations*. 2022.